

Consultas Avançadas

Banco de dados de uma concessionária: reservas, manutenções, multas e avaliações.



Augusto



André



Daniel



Guilherme

O estabelecimento

A concessionária registra quatro eventos no banco: a **reserva** do veículo, a **manutenção** da frota, as **multas** cometidas durante o período de posse e a **avaliação** deixada pelo cliente.

RESERVA

20

registros

MANUTENÇÃO

13

registros

MULTA

10

registros

AVALIAÇÃO

12

registros

ONDE ESTAMOS

As etapas do trabalho

ETAPA 02 ✓

Modelo Conceitual

Entidades, relacionamentos e cardinalidades no ER.

ETAPA 03 ✓

Modelo Lógico

Tabelas, chaves primárias e estrangeiras.

ETAPA 04 ✓

Implementação SQL

DDL e DML no PostgreSQL.

ETAPA 05

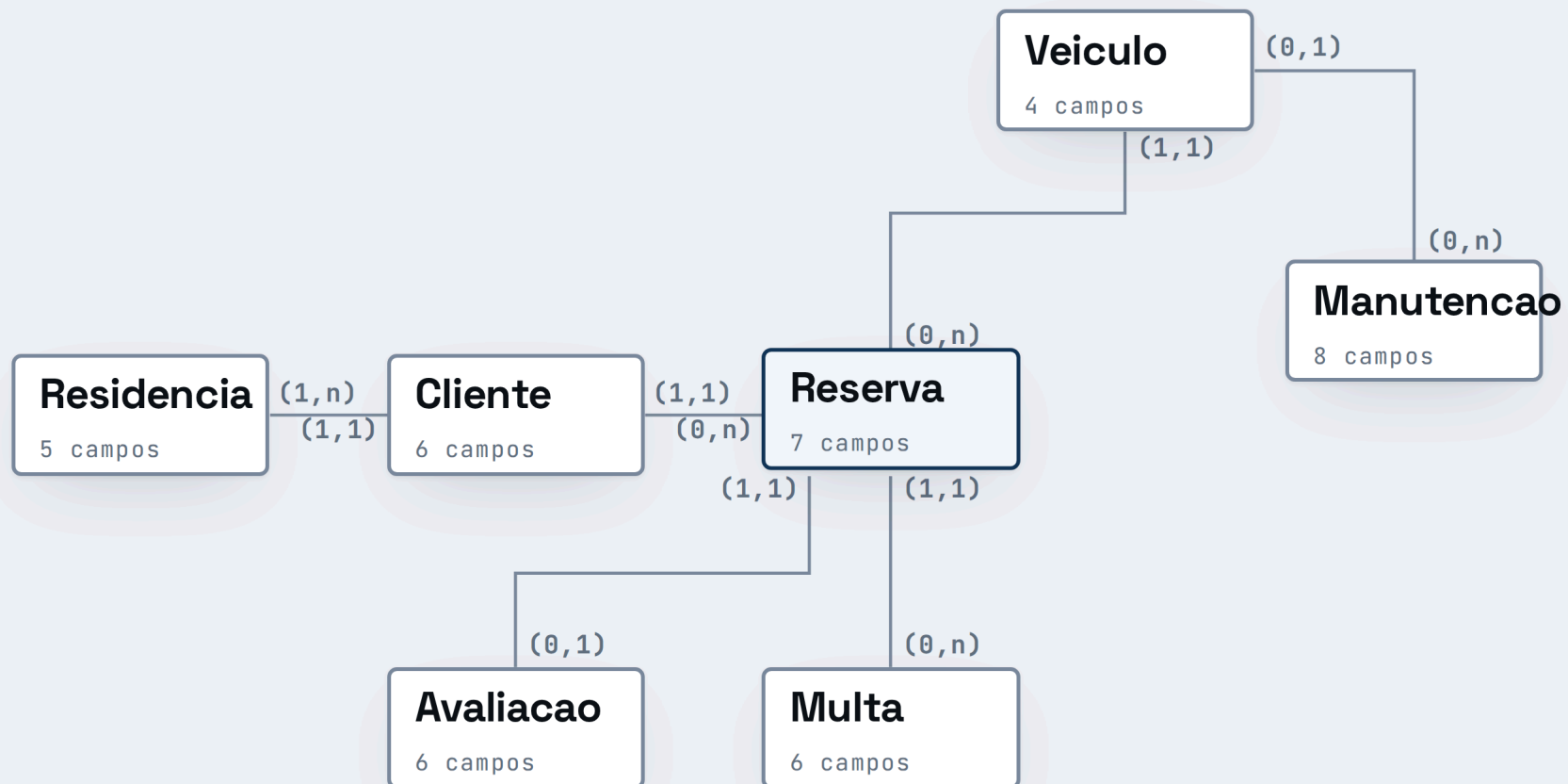
Consultas Avançadas

Quatro consultas analíticas, uma por integrante.

Modelo lógico

CLIQUE

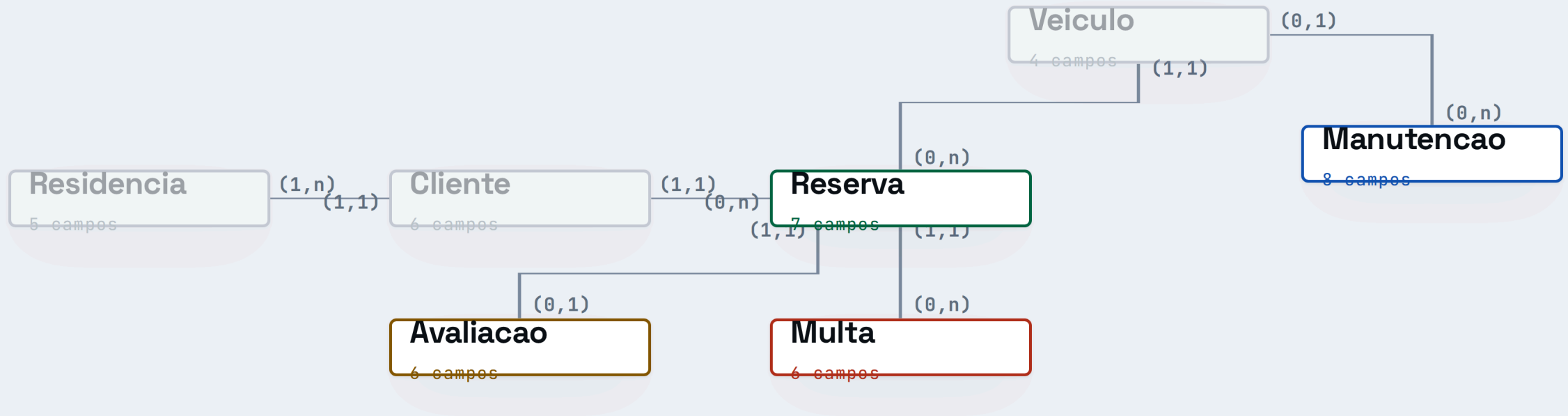
NAS TABELAS



Reserva

PK	id
	inicio
	fim
	retirada
	status
FK	fk_Cliente_cpf
FK	fk_Veiculo_placa NOVO

Divisão do grupo



MULTA • INNER JOIN • COUNT

Augusto

MANUTENÇÃO • LEFT JOIN • SUM

André

AVALIAÇÃO • RIGHT JOIN • AVG

Daniel

RESERVA • FULL OUTER • MAX E MIN

Guilherme

AUGUSTO · EVENTO MULTA

PERGUNTA DE NEGÓCIO

Quais clientes acumulam duas ou mais multas?

FUNÇÃO DE AGREGAÇÃO · COUNT



Três tabelas

MULTA		10 LINHAS
ID	VALOR	RESERVA
1	293	2
2	195	4
3	880	12
4	130	7

RESERVA		20 LINHAS
ID	FK_CLIENTE_CPF	
2	22233344455	
4	22233344455	
12	22233344455	
7	66677788899	

CLIENTE		12 LINHAS
CPF	NOME	
22233344455	Carlos Eduardo Lima	
66677788899	Gabriel Fonseca	
88899900011	Igor Salgado	
11122233344	Ana Beatriz Moraes	

Multa tem o valor mas não o nome. **Cliente** tem o nome mas não as multas. E a multa nem guarda o CPF — **Reserva é a ponte**. Responder a pergunta exige ler as três ao mesmo tempo, e é isso que o JOIN faz.

```
SELECT c.nome, c.contato,  
       COUNT(m.id)      AS total_multas,  
       COUNT(DISTINCT r.id) AS reservas  
FROM Multa m  
INNER JOIN Reserva r ON m.fk_Reserva_id = r.id  
INNER JOIN Cliente c ON r.fk_Cliente_cpf = c.cpf  
GROUP BY c.cpf, c.nome, c.contato  
HAVING COUNT(m.id) >= 2  
ORDER BY total_multas DESC;
```

SELECT

COUNT(m.id) conta as multas. COUNT(DISTINCT r.id) conta em quantas reservas diferentes elas aconteceram.

FROM MULTA

A consulta parte da multa, não do cliente. O relatório é sobre quem tem multa.

INNER JOIN

Duas junções encadeadas. Cliente sem multa não aparece nem zerado — é a interseção das três tabelas.

GROUP BY

Um grupo por cliente. As três multas de Carlos passam a ser uma linha.

HAVING

Filtra os **grupos já calculados** — quem tem uma multa só sai do relatório. Não dá para usar WHERE aqui: quando ele roda, o COUNT ainda nem foi calculado.

ORDER BY

Do maior número de multas para o menor.

Resultado da execução

REINCIDENTES			3 LINHAS	
CLIENTE	CONTATO	TOTAL_MULTAS	RESERVAS	
Carlos Eduardo Lima	(11) 97654-3210	3	3	
Gabriel Fonseca	(11) 93210-9876	2	2	
Igor Salgado	(11) 91098-7654	2	2	

CORTE DO HAVING

2 multas



HAVING COUNT(m.id) ≥ 2

Arraste e veja o filtro agir sobre os grupos.
Em 1, todo cliente com multa aparece e o relatório perde a utilidade.

AUGUSTO · LEITURA DO RESULTADO

7 de 10

multas registradas vêm de **três clientes**. Os outros sete clientes somam três.

USO: REVISAR CAUÇÃO E COBERTURA ANTES DE APROVAR NOVA RESERVA

PERGUNTA DE NEGÓCIO

Quais veículos custaram
R\$ 2.000 ou mais em
manutenção, e quais
nunca foram à oficina?

FUNÇÃO DE AGREGAÇÃO · SUM

Duas tabelas

VEICULO12 LINHAS		
PLACA	MARCA	MODELO
ABC1D23	Volkswagen	Nivus
BRA2E19	Chevrolet	Onix
CDE3F45	Fiat	Pulse
DEF4G56	Toyota	Corolla

MANUTENCAO13 LINHAS		
ID	FK_VEICULO_PLACA	VALOR
1	ABC1D23	1800
2	ABC1D23	950
3	DEF4G56	1200
4	DEF4G56	2400
5	DEF4G56	1500

Quatro veículos não aparecem nenhuma vez em Manutencao. São eles que decidem qual junção usar.

```
SELECT v.placa, v.marca, v.modelo,  
       COUNT(m.id)          AS qtd_manutencoes,  
       COALESCE(SUM(m.valor), 0) AS custo_total  
FROM Veiculo v  
LEFT JOIN Manutencao m ON v.placa = m.fk_Veiculo_placa  
GROUP BY v.placa, v.marca, v.modelo  
HAVING COALESCE(SUM(m.valor),0) >= 2000  
       OR SUM(m.valor) IS NULL  
ORDER BY custo_total DESC;
```

SELECT

COALESCE troca o NULL do SUM por 0, para o relatório não mostrar campo vazio.

FROM VEICULO

Veiculo é a tabela da esquerda. É ela que será preservada inteira.

LEFT JOIN

Sem manutenção, o veículo continua na lista e o lado direito vira NULL. Com INNER JOIN, quatro veículos sumiriam.

GROUP BY

Um grupo por veículo. As três manutenções do Corolla viram uma linha.

HAVING

A primeira condição pega os carros. A segunda mantém os que nunca foram à oficina, que sem ela seriam eliminados.

ORDER BY

Do maior custo para o menor.

Resultado da execução

BRASIL

LEFT JOIN

CUSTO POR VEÍCULO				8 LINHAS
PLACA	MARCA	MODELO	QTD_MANUTENCOES	CUSTO_TOTAL
DEF4G56	Toyota	Corolla	3	5100
GHI7J89	Jeep	Renegade	2	4090
JKL0M12	Ford	Ranger	2	3550
ABC1D23	Volkswagen	Nivus	2	2750
CDE3F45	Fiat	Pulse	0	0
EFG5H67	Honda	Civic	0	0
HIJ8K90	Renault	Kwid	0	0
LMN2034	Citroen	C3	0	0

CORTE DO HAVING

R\$ 2.000



trocar por INNER JOIN

R\$ 15.490

saíram em **quatro dos doze veículos** — 82% de tudo que a frota gastou em oficina. Outros quatro nunca entraram.

USO: PRIORIZAR A RENOVAÇÃO PELOS QUATRO MAIS CAROS

PERGUNTA DE NEGÓCIO

Qual a **nota média por canal**, e quantas reservas não foram avaliadas?

FUNÇÃO DE AGREGAÇÃO · AVG

Duas tabelas

AVALIACAO			12 LINHAS
ID	NOTA	PLATAFORMA	RESERVA
1	9	Google Maps	1
3	7	Instagram	3
5	5	WhatsApp	5
8	9	Site Proprio	8

RESERVA		20 LINHAS
ID	STATUS	
1	Concluida	
3	Concluida	
15	Concluida	
18	Pendente	
20	Pendente	

São 20 reservas para 12 avaliações. **Oito reservas não têm avaliação nenhuma** — as de id 15, 18 e 20, entre outras.

```
SELECT COALESCE(a.plataforma, '(sem avaliacao)') AS canal,  
       COUNT(r.id) AS reservas,  
       ROUND(AVG(a.nota), 2) AS nota_media  
FROM Avaliacao a  
RIGHT JOIN Reserva r ON a.fk_Reserva_id = r.id  
GROUP BY a.plataforma  
HAVING AVG(a.nota) < 8.5  
       OR AVG(a.nota) IS NULL  
ORDER BY nota_media NULLS LAST;
```

SELECT

Quando não há avaliação, plataforma é NULL. O COALESCE dá nome a esse grupo.

FROM AVALIACAO

A avaliação está à esquerda, mas não é ela que precisa ser preservada.

RIGHT JOIN

Preserva **Reserva**, que está à direita. Toda reserva conta, mesmo sem avaliação. É o espelho do LEFT JOIN: bastaria inverter a ordem das tabelas.

GROUP BY

Um grupo por plataforma. As reservas sem avaliação caem todas no grupo NULL.

HAVING

Canais abaixo de 8,5. A segunda condição mantém o grupo sem avaliação, cujo AVG é NULL.

ORDER BY

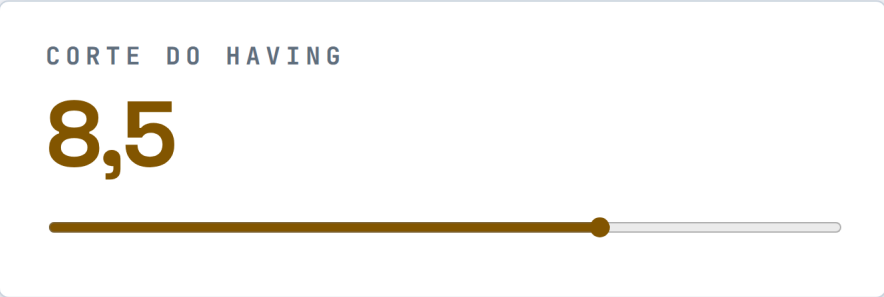
NULLS LAST joga o grupo sem nota para o fim.

Resultado da execução

BRASIL

RIGHT JOIN

SATISFAÇÃO POR CANAL			4 LINHAS
CANAL	RESERVAS	NOTA_MEDIA	
WhatsApp	2	5,50	
Instagram	3	7,00	
Site Proprio	3	8,00	
(sem avaliacao)	8	NULL	



O grupo **(sem avaliacao)** nunca sai, em nenhuma posição do controle: o AVG dele é NULL e quem o segura é o OR AVG(a.nota) IS NULL.

8 de 20

reservas não geraram avaliação nenhuma.
Nenhum ranking de nota mostra esse grupo —
só o RIGHT JOIN.

USO: PEDIR AVALIAÇÃO ATIVAMENTE NA DEVOLUÇÃO DO VEÍCULO

PERGUNTA DE NEGÓCIO

Quais veículos estão
parados, e quais reservas
estão **sem veículo**
alocado?

FUNÇÕES DE AGREGAÇÃO · MAX E MIN

Duas tabelas

VEICULO12 LINHAS	
PLACA	MODELO
BRA2E19	Onix
HIJ8K90	Kwid
KLM1N23	208
LMN2034	C3

RESERVA20 LINHAS			
ID	INICIO	FIM	FK_VEICULO_PLACA
3	2025-10-15	2025-10-22	BRA2E19
5	2025-11-13	2025-11-20	HIJ8K90
18	2026-07-06	2026-07-13	NULL
20	2026-08-10	2026-08-17	NULL

Os dois lados têm sobra: veículos que ninguém reservou, e reservas com fk_Veiculo_placa em NULL, ainda sem carro designado.

```
SELECT COALESCE(v.placa, '(sem veiculo alugado)') AS veiculo,  
       COUNT(r.id) AS reservas,  
       MIN(r.inicio) AS primeira,  
       MAX(r.fim)   AS ultima  
FROM Veiculo v  
FULL OUTER JOIN Reserva r ON v.placa = r.fk_Veiculo_placa  
GROUP BY v.placa, v.modelo  
HAVING MAX(r.fim) IS NULL           -- nunca reservado  
       OR v.placa IS NULL           -- reserva sem carro  
       OR MAX(r.fim) < DATE '2026-01-01' -- parado  
ORDER BY ultima NULLS FIRST;
```

SELECT

MIN(inicio) e MAX(fim) dão o primeiro e o último dia em que o carro rodou.

FROM VEICULO

Aqui a ordem das tabelas não importa: o FULL OUTER preserva os dois lados.

FULL OUTER JOIN

Único que traz as duas sobras juntas. O PostgreSQL suporta direto; no MySQL seria preciso simular com UNION.

GROUP BY

Um grupo por veículo. As reservas sem carro caem todas no grupo NULL.

HAVING

Três condições, um problema cada. **Sem a segunda linha** o HAVING eliminaria as reservas sem carro — elas têm data recente, não são NULL nem antigas — e o FULL OUTER perderia metade da função.

ORDER BY

NULLS FIRST põe os nunca reservados no topo.

A mesma consulta com cada junção

- INNER
- LEFT
- RIGHT
- FULL OUTER

RESULTADO				5 LINHAS	
VEICULO	MODELO	RESERVAS	PRIMEIRA	ULTIMA	
KLM1N23	208	0	NULL	NULL	
LMN2034	C3	0	NULL	NULL	
BRA2E19	Onix	1	2025-10-15	2025-10-22	
HIJ8K90	Kwid	1	2025-11-13	2025-11-20	
(sem veiculo alugado)	---	3	2026-07-06	2026-08-17	

5

Unico que mostra os dois problemas ao mesmo tempo.

2 · 2 · 3

dois veículos nunca foram reservados, **dois** estão parados desde 2025 e **três** reservas seguem sem carro alocado — a primeira começa em 06/07.

USO: ALOCAR OS PARADOS NAS TRÊS RESERVAS PENDENTES

Console SQL

NO NAVEGADOR

Augusto

André

Daniel

Guilherme

EXPLAIN

o esquema

CONSULTA

CTRL + ENTER

```
-- AUGUSTO · Multa · INNER JOIN · COUNT
SELECT c.nome AS cliente, c.contato,
       COUNT(m.id) AS total_multas,
       COUNT(DISTINCT r.id) AS reservas
FROM Multa m
INNER JOIN Reserva r ON m.fk_Reserva_id = r.id
INNER JOIN Cliente c ON r.fk_Cliente_cpf = c.cpf
```

EXECUTAR

Executado

RESULTADO

3 LINHAS · 2 MS

CLIENTE	CONTATO	TOTAL_MULTAS	RESERVAS
Carlos Eduardo Lima	(11) 97654-3210	3	3
Gabriel Fonseca	(11) 93210-9876	2	2
Igor Salgado	(11) 91098-7654	2	2

Não é tabela pronta: é o **PostgreSQL compilado para o navegador**, com a mesma carga do `etapa05_completo.sql`. Escreva qualquer consulta e execute.

Cobertura da entrega

QUATRO JUNÇÕES, UMA POR INTEGRANTE					4 DE 4
INTEGRANTE	EVENTO	JUNÇÃO	FUNÇÃO	PERGUNTA DE NEGÓCIO	
Augusto	Multa	INNER JOIN	COUNT	Clientes com duas ou mais multas	
André	Manutenção	LEFT JOIN	SUM	Veículos caros de manter e os que nunca foram à oficina	
Daniel	Avaliação	RIGHT JOIN	AVG	Nota por canal e reservas sem avaliação	
Guilherme	Reserva	FULL OUTER	MAX · MIN	Veículos parados e reservas sem carro	

FUNÇÕES ✓

COUNT · SUM · AVG · MAX · MIN

5 de 5

EM TODAS ✓

GROUP BY · HAVING

4 de 4 consultas

O que aprendemos

01 • MODELAGEM

A FK que faltava

A relação Veículo–Reserva estava no modelo conceitual, mas não tinha sido implementada no DDL. O erro só apareceu quando tentamos responder qual carro foi reservado.

02 • JUNÇÕES

NULL é o que separa LEFT de FULL

Quando a chave estrangeira é obrigatória, LEFT JOIN e FULL OUTER JOIN devolvem o mesmo. O lado direito só sobra se a FK aceitar NULL — por isso a consulta do Guilherme usa Reserva.

03 • HAVING

O filtro pode desfazer a junção

Sem a condição `v.placa IS NULL`, o HAVING descartava as reservas sem carro que o FULL OUTER tinha acabado de trazer. A junção certa com o filtro errado dá resposta errada.

Entregáveis

SCRIPT

etapa05_completo.sql

DDL, DML e as 4 consultas

PRINTS

Execução no SGBD

PostgreSQL 16

SLIDES

Este site

29 slides

Abrir o script SQL